BRAUDE
College of Engineering, karmiel

Capstone Project Phase B

Interactive Mathematical Proof Verification System

24-2-R-15

Authors: Daniel Armaganian, Tzahi Bakal

Supervisors: Dr. Dan Lemberg, Mrs. Elena Kramer

Table of Contents

Abstract. Recent advancements in artificial intelligence, particularly large language models (LLMs) such as ChatGPT, have demonstrated their capacity to generate mathematical proofs with remarkable skill. Nevertheless, ChatGPT is not yet capable of verifying the accuracy of its proofs, which presents a major obstacle to their integration into formal mathematical studies. This project addresses this challenge by developing an interactive proof verification system. Our framework enables the verification of proofs generated by ChatGPT using the formal proof language Agda. The system facilitates seamless interaction between ChatGPT and the formal proof environment through OpenAi's API, ensuring that proofs are both generated and rigorously validated. This integration not only enhances the reliability of AI-assisted mathematical proofs but also accelerates mathematical research and significantly improves the trustworthiness of AI-generated results. By bridging the gap between proof generation and verification, this project lays the groundwork for more dependable AI applications in mathematics and beyond.

Keywords: Artificial Intelligence · Large Language Models · ChatGPT · Formal Verification · Formal Proof Languages · Agda.

1 Introduction

In current years, the sphere of artificial intelligence has witnessed superb advancements, specifically within the domain of LLMs. These models, exemplified by using structures like ChatGPT, have validated an outstanding capability to interact in complicated reasoning obligations, together with the method of mathematical proofs [1], [5]. However, a difficulty persists: at the same time that ChatGPT can generate proofs, it lacks the capability to verify the correctness of the proofs it generates [2], [5]. This gap between proof generation and verification presents a challenge in fields where accuracy is crucial. As LLMs are increasingly adopted in research, it becomes critical to develop methods that ensure the trustworthiness of their outputs.

This project addresses this gap by introducing an interactive proof verification system that bridges the creative potential of LLMs with the rigorous foundation of formal mathematics. Building on Per Martin-Löf and Sambin's [3] concept of intuitive types and Vladimir Voevodsky's [4] concept of grounded Homotopy types, we developed a framework capable of interacting with OpenAi's 4o-model API to solve and verify mathematical proofs. By combining the power of LLMs with formal verification, our system provides an essential tool for ensuring mathematical correctness while harnessing the potential and adaptability of LLMs.

The significance of this task cannot be overstated. As LLMs are increasingly included into mathematical and scientific research, it is crucial to guarantee or at least significantly improve the accuracy of their results. Beyond mathematics, such advancements hold the potential to impact other fields requiring formal reasoning, such as software verification, automated theorem proving, and even decision-making in critical systems. By integrating between human intuition, artificial intelligence, and formal verification, our system offers a robust solution that accelerates mathematical research while simultaneously enhancing the trustworthiness of AI-assisted mathematical discoveries.

Our method is based on Voevodsky's Homotopy type theory [4], which provides tools for managing higher-dimensional structures and equalities, and Martin-Löf and Sambin's intuitionistic type theory [3], which provides a framework for constructive reasoning. By implementing these theories within formal proof languages like Agda, we created a framework where proofs generated by ChatGPT can be validated effectively. Furthermore, our work introduces a scalable architecture, adaptable for future extensions into broader areas of formal logic and reasoning.

The remainder of this paper proceeds as follows. First, we will explore the technical details of our system, providing an overview of the relevant type theories. We will then describe the architecture of our verification system, including the design of the API for communication with OpenAi's 4o-model. We then present the results from our research, highlighting the efficacy of the proposed system, and conclude with a discussion of challenges encountered and directions for future work.

2 Literature Review

2.1 ChatGPT as an Expert Tool

Azaria, Azoulay, and Reches (2024) discuss the potential of ChatGPT as a powerful assistant for experts, highlighting its capability to generate complex solutions across various subjects. However, they emphasize that its outputs may require validation to ensure accuracy, pointing to the need of integrating some kind of a verification system when deploying LLMs in fields like mathematics [1]. This observation shows the importance of our system, which seeks to develop a mechanism that ensures the reliability of AI-generated proofs.

2.2 LLMs and Software Verification

Janßen, Richter, and Wehrheim (2024) explore the application of LLMs like ChatGPT in the domain of software verification. They find that while ChatGPT can produce valid and useful invariants that assist formal verifiers, the tool alone is insufficient for full verification of logical correctness. The study highlights the necessity of integrating these LLM-generated outputs with robust formal verification tools and human oversight [2]. This finding is relevant to our research, as it underscores the limitations of relying solely on LLMs for verifying the correctness of mathematical proofs.

2.3 Foundational Theories in Proof Verification

Our approach draws on the foundational theories of Intuitionistic Type Theory (ITT) and Homotopy Type Theory (HoTT), which provide the theoretical foundations for constructing and verifying proofs.

Martin-Löf and Sambin's (1984) ITT offers a framework for constructive reasoning, where mathematical proofs are treated as algorithms that must be explicitly constructed [3]. This aligns with the need for a verification system that can confirm the correctness of AI-generated proofs through constructive methods.

Voevodsky's HoTT introduces tools for reasoning about higher-dimensional structures and equalities, offering a robust framework for formalizing complex mathematical objects [4]. By integrating these theories into our system, we aim to create a verification

framework that not only validates proofs but also accommodates the intricate structures that AI-generated proofs may involve.

2.4 Agda in Formal Verification

The implementation of these type theories in practical proof assistants has led to the development of formal proof languages such as Agda. Stump (2016) emphasizes the role of Agda, a dependently typed programming language, in formal verification. Agda's support for constructive proofs and its ability to represent complex mathematical structures with precision [9], make it an ideal choice for our proposed verification system. By employing Agda, our system will be able to formalize and verify the correctness of proofs generated by ChatGPT, ensuring their reliability and consistency.

```
{- Definition of natural numbers (ℕ) with zero and successor constructors -}
data ℕ : Set where
  zero : ℕ
  succ : ℕ → ℕ

{- Definition of equality (≡) for natural numbers with reflexivity constructor (n≡n) -}
data _≡_ : ℕ → ℕ → Set where
  n≡n : {n : ℕ} → n ≡ n

{- Definition of addition for natural numbers -}
_+_ : ℕ → ℕ → ℕ
n + zero = n
n + (succ m) = succ (n + m)

{- Proves that if m == n, then m + 1 == n + 1 -}
m≡n⇒sm≡sn : {m n : ℕ} → m ≡ n → (succ m) ≡ (succ n)
m≡n⇒sm≡sn n≡n = n≡n

{- Transitive relation. Helps for chaining equations into the final result -}
k≡m∧m≡n⇒k≡n : {k m n : ℕ} → k ≡ m → m ≡ n → k ≡ n
k≡m∧m≡n⇒k≡n n≡n n≡n = n≡n

{- Proof that zero + n ≡ n + zero for any natural number n -}
z+n≡n+z : (n : ℕ) → (zero + n) ≡ (n + zero)
z+n≡n+z zero = n≡n
z+n≡n+z (succ n) = m≡n⇒sm≡sn (z+n≡n+z n)

{- Proves that for any number m and n the equality succ m + n = succ (m +n) holds -}
sm+n≡s[m+n] : (m n : ℕ) → (succ m + n) ≡ succ (m + n)
sm+n≡s[m+n] m zero = n≡n
sm+n≡s[m+n] m (succ n) = m≡n⇒sm≡sn (sm+n≡s[m+n] m n)

{- Main proof: Commutativity of addition -> or any a and b, the equality a + b = b + a holds.-}
m+n≡n+m : (m n : ℕ) → (m + n) ≡ (n + m)
m+n≡n+m zero n = z+n≡n+z n
m+n≡n+m (succ m) n = k≡m∧m≡n⇒k≡n (sm+n≡s[m+n] m n) (m≡n⇒sm≡sn (m+n≡n+m m n))
```

Figure 1. Commutativity proof of addition in Agda. Created by Carbon https://carbon.now.sh/

2.5 LLMs in Mathematical Problem Solving

Plevris, Papazafeiropoulos, and Rios (2023) conducted a comparative study on the performance of various LLMs, including ChatGPT, in solving mathematical and logical problems. Their research highlights the variability in the logical clearness of solutions provided by these models, showing the need for verification systems [5]. While ChatGPT can generate mathematical questions and explanations across various difficulty levels, it struggles with consistently producing correct proofs, especially for complex problems. Similarly, Shakarian et al. [7] conducted an independent evaluation of ChatGPT's capabilities in solving mathematical word problems, highlighting significant challenges when faced with complex problems. These studies support our approach of integrating a formal verification framework with ChatGPT to ensure the correctness of its generated proofs.

2.6 Unifying Type Theory and Formal Verification

Pelayo and Warren (2014) explore Homotopy Type Theory and its univalent foundations, providing a unified approach to reasoning about mathematical structures. Their work highlights the potential of HoTT to serve as a robust foundation for formal verification, particularly in the context of complex mathematical proofs [4]. This theoretical foundation is critical to our project, as it informs the development of a verification system that leverages both intuitionistic and homotopy type theories to ensure the correctness of AI-generated proofs.

3 Background

Our work builds upon two fundamental theories in type theory: Intuitionistic Type Theory (ITT) and Homotopy Type Theory (HoTT). We begin by discussing ITT, followed by HoTT, the use of the Agda programming language, and finally, our system's API.

3.1 Intuitionistic Type Theory

Intuitionistic Type Theory, developed by Per Martin-Löf and Sambin, represents a significant advancement in the formalization of mathematical reasoning. The primary goal of ITT was to construct a formal system that could represent informal mathematical reasoning more accurately than previous foundational systems [3].

A key feature of ITT is the distinction between propositions and judgments. Propositions are statements that can be combined using logical operations and can be true or false, while judgments are assertions about the truth of propositions. This distinction forms the basis for the system's rules of inference, which are justified by explaining how conclusions can be drawn from premises known to be true.

ITT introduces four fundamental forms of judgment:

1. A is a set
2. A and B are equal sets
3. a is an element of set A
4. a and b are equal elements of set A

In this system, sets are defined by specifying rules for forming canonical elements and equal canonical elements. An element of a set is conceptualized as a method or program that yields a canonical element of that set when executed. One of the distinguishing features of ITT is its intuitionistic approach to propositions. Rather than defining propositions in terms of truth values, ITT defines them in terms of what constitutes a proof of the proposition. This aligns with the intuitionistic interpretation of logical operations and allows for a more constructive approach to mathematics.

The theory pays particular attention to specific set operations, including the Cartesian product, disjoint union, and natural numbers. These operations and their associated rules are used to interpret various logical connectives and quantifiers. Notably, ITT provides a proof for the axiom of choice within its framework, demonstrating the system's expressive power.

To extend the system beyond finite types, ITT introduces the concept of universes. These allow for the construction of transfinite types, which are necessary for dealing with certain concepts in category theory that cannot be adequately captured with just finite types.

Overall, ITT aims to provide a rich formal system that serves a dual purpose: as a rigorous foundation for mathematics and as a practical programming language. This dual nature makes ITT particularly relevant to our work, as it bridges the gap between theoretical foundations and practical implementations in computer science.

3.2 Homotopy Type Theory

Homotopy Type Theory (HoTT) represents a combination of abstract homotopy theory and type theory. It emerged from the discovery of deep connections between Martin-Löf and Sambin's constructive type theory and abstract homotopy theory [4].

The key insight of HoTT is the interpretation of types as spaces (or homotopy types) and terms of a type as points in that space.

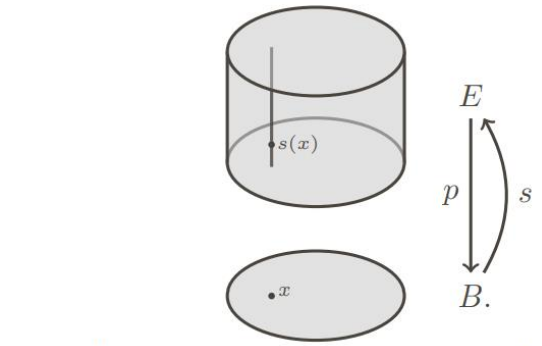

Figure 2. Illustration of how dependent types in type theory can be interpreted geometrically as fibrations in homotopy theory [4].

A central concept in HoTT is the Univalence Axiom, introduced by Vladimir Voevodsky (2014). This axiom states that identity between types is equivalent to equivalence of types. Formally: $(A = B) \simeq (A \simeq B)$ where $\simeq$ denotes equivalence. This principle allows for more flexible reasoning about equality.
"In other words, identity is equivalent to equivalence. In particular, one may say that 'equivalent types are identical'. [6]

HoTT also introduces higher inductive types (HITs), which allow for the direct encoding of many important spaces and constructions from homotopy theory within the type theory itself.

The univalent perspective, proposes adopting HoTT as a foundation for mathematics, offering several key advantages. It provides a constructive foundation that aligns with homotopy theoretic concepts, enabling direct formal verification of proofs in varies proof assistants such as Agda. Furthermore, this approach yields fresh insights and techniques that benefit both type theory and homotopy theory. By linking the logic of type theory with the geometric intuitions of homotopy theory, HoTT creates a powerful framework that not only enhances our understanding of mathematical structures but also facilitates the development of machine-checkable proofs, potentially revolutionizing how we approach mathematical reasoning and verification.

3.3 Agda

In this project, we utilized Agda, a dependently typed programming language rooted in Martin-Löf and Sambin's ITT [9]. This theoretical foundation provides a framework for constructive reasoning, where mathematical proofs are approached as algorithms that need to be explicitly formulated and verified.

Agda is a dependently typed programming language primarily used for writing and verifying formal proofs in a functional programming setting. As Sitnikovski [8] explains, dependent types provide a powerful way to encode program proofs, offering a robust foundation for formal verification in Agda.
As an advanced tool for verified programming, Agda offers a unique approach where the correctness of programs is ensured through mathematical proofs written in the language itself. The proofs are checked by Agda's compiler, which verifies that the program adheres to its specifications across all possible inputs.

Agda is grounded in constructive logic, where types are viewed as logical propositions, and programs that inhabit these types serve as proofs of the propositions. This approach is based on the Curry-Howard correspondence, a fundamental concept in type theory that equates types with propositions and programs with proofs.

Agda's type system is highly expressive, supporting features like dependent types, where types can depend on values. This allows for precise specifications of program properties, such as the length of a vector being encoded in its type. Additionally, Agda supports both internal and external verification of functions, offering a framework for proving the correctness of complex algorithms and data structures.

The language's strong emphasis on pure functional programming, where functions behave like mathematical functions without side effects, aligns well with the goals of verified programming. Agda also enforces termination, ensuring that all programs eventually produce a result, crucial for maintaining logical consistency in proofs.

Agda's interactive development environment, allows users to write, type-check, and interactively develop proofs and programs. This environment supports Unicode, enabling the use of mathematical symbols directly in code, which enhances the readability and expressiveness of proofs.

Overall, Agda represents a powerful tool for researchers and developers interested in formal methods, providing a platform to explore and apply advanced concepts in type theory and functional programming.

3.3.1 More on Agda

Agda's power as a proof assistant and programming language stems from several key features:

Inductive Types: Agda allows the definition of inductive data types, which are fundamental for representing recursive structures.

```
data ℕ : Set where
  zero : ℕ
  suc : ℕ → ℕ
```

Figure 3. Agda code of inductive definition of the type N for the Peano natural numbers [9].

Pattern Matching and Recursion: Agda uses pattern matching and structural recursion for function definitions, allowing for clear and concise code.

```
_+_ : ℕ → ℕ → ℕ
zero  + n = n
suc m + n = suc (m + n)
```

Figure 4. Agda code of recursive definition of addition [9].

3.3.2 Key Features of Agda Used in This Project

Agda's strength as both a proof assistant and a programming language comes from its core features: dependent types, which enable precise specifications by allowing types to depend on values (such as encoding a vector's length in its type); inductive types, which allow for the formal definition of recursive structures like Peano natural numbers; and pattern matching and recursion, which facilitate concise and logically sound function definitions. Additionally, Agda's interactive development environment supports real-time proof checking. It was chosen over other proof assistants like Idris [8] due to its constructive approach, strong type system, and seamless integration with functional programming.

3.4 Our System's API and Contributions

Our system features a dedicated API that enables interaction between ChatGPT's proof generation capabilities and Agda's formal verification framework. This API serves as an intermediary, ensuring that AI-generated proofs can be automatically translated, refined, and verified within Agda.

The API accepts natural language input, translating it into a semi-formal proof structure that is subsequently validated by Agda. The system is designed for iterative refinement, where failed proofs are analyzed, and feedback is provided to the proof generation module, enabling continuous improvement until convergence is achieved. This integration marks a significant step toward making AI-generated proofs more reliable and creates opportunities for future work in automated theorem proving and AI-assisted formal reasoning.

4 Research/Engineering Process

4.1 Problem Statement

The primary challenge addressed in this research is the inability of ChatGPT to verify the mathematical proofs they generate. While these LLMs are capable of producing proofs with a high degree of complexity, there remains no mechanism to validate their correctness independently. To address this, we developed an interactive proof verification system that integrates Intuitionistic Type Theory and Homotopy Type Theory. Our system employs Agda as a formal proof verification tool and OpenAI's ChatGPT-4o API for proof generation. By structuring the interaction between these components through an iterative refinement process, we significantly improve the correctness and reliability of AI-generated mathematical proofs.

4.1.1 Theoretical Foundations and Research Methodology

Our research builds upon two major type theories: Intuitionistic Type Theory (ITT), developed by Per Martin-Löf, which emphasizes constructive mathematics where proofs are computationally interpretable, and Homotopy Type Theory (HoTT), introduced by Vladimir Voevodsky, which extends type theory with higher-dimensional structures and the Univalence Axiom, equating type equivalence with equality. ITT serves as the logical basis for Agda, ensuring that all proofs are constructively verifiable, while HoTT allows for flexible reasoning about equivalences. Our research follows an iterative development methodology, combining theoretical proof construction with practical AI-driven verification. Testing was conducted through formal analysis, computational validation, and expert review, measuring proof convergence, syntactic correctness, and logical validity.

4.1.2 Study Limitations

While our system effectively integrates ChatGPT-4o and Agda to verify mathematical proofs, it has several limitations. One key constraint is non-convergence in complex mathematical statements, where the iterative refinement process may not always produce a verified proof within a reasonable number of iterations. Additionally, ChatGPT-4o's reasoning capabilities have inherent limitations, as LLMs can sometimes misinterpret Agda's feedback, leading to redundant refinements.

Computational efficiency is another challenge, as iterative proof generation and verification require significant API calls, making large-scale implementation costly. Lastly, the system is currently restricted to Agda, and future work should explore integration with other proof assistants to broaden applicability.

4.2 System Architecture

The proof verification system consists of three main components:

1. User's mathematical prompt in natural language.
2. ChatGPT-4o API for generating Agda code as proofs.
3. Agda for verifying proofs.

By combining those three components, we create smooth communication between a LLM and a formal language like Agda, that automates the process of verifying a generated proof, enhancing efficiency and reliability in LLMs.

4.3 Proof Refinement Algorithm

The main part of our solution is integrating ChatGPT-4o and Agda within an iterative proof verification system. ChatGPT-4o generates mathematical proofs, while Agda verifies their correctness.

```
function refine_proof(proof)
    attempted_proofs = set()

    for i in range(iteration_limit) do
        feedback = agda_verify(proof)
        if feedback is valid then
            return proof if chatgpt_validate_solution(proof) else None
        end if

        if proof in attempted_proofs then
            proof = chatgpt_refine(feedback, change_direction=True)
        else
            proof = chatgpt_refine(feedback)
        end if

        attempted_proofs.add(proof)
    end for

    return None
end function
```

Figure 5. Pseudocode of the algorithm. Created by Carbon https://carbon.now.sh/

The algorithm starts with a user-submitted proof request, which ChatGPT-4o processes to generate an initial proof. Agda then verifies the proof, and if it fails, Agda's feedback is sent back to ChatGPT-4o for refinement. Before resubmitting, the system checks for redundant proof attempts to prevent repeated failures. Once Agda successfully verifies a proof, an additional validation step prompts ChatGPT-4o to confirm whether the proof correctly solves the original problem. This process repeats until the proof is valid or the time limit is reached, with the result displayed to the user.

4.4 Challenges

During implementation, we encountered several challenges that required improvements to our system to ensure efficiency and accuracy.

4.4.1 ChatGPT-4o Mini Performance Issues

Initially, our system was developed ChatGPT-4o Mini as the API, to optimize costs, but it failed to consistently generate valid Agda proofs and lost context very quickly. The complexity of Agda's syntax and type system required us to upgrade to the full GPT-4o model, to ensure better accuracy and reliability, and perhaps even higher models in the future.

4.4.2 Non-Convergence in Proof Refinement

Some proofs failed to converge within the allocated iteration limit, particularly for highly abstract mathematical statements. Agda's feedback was often ambiguous, which made it difficult for ChatGPT-4o to refine proofs effectively. To address this, we implemented structured hints and optimized prompt engineering to provide more precise feedback to the API.

Additionally, we added a mechanism to detect redundant proof attempts, which helps prevent infinite refinement loops by instructing ChatGPT-4o to adjust its approach when a previously failed proof is attempted again. While this does not completely eliminate the issue, it significantly reduces unnecessary iterations.

4.4.3 Efficiency and API Optimization

Without optimizations, our system became computationally expensive due to repeated API calls. To improve performance, we moved all of the API calls to separate threads, optimized prompts to reduce iteration numbers, which allowed for the system to remain smooth at all times.

4.4.4 Agda's Strict Syntax and GPT-4o Adaptation

Unlike mainstream programming languages, Agda enforces a highly structured syntax and strict type rules. ChatGPT-4o model often struggled to fully adhere these constraints, which required iterative tuning and custom error handling mechanisms to correctly guide the API towards proof verification convergence.

4.4.5 Ensuring Proof Correctness Beyond Agda Verification

Another challenge we encountered was the realization that Agda verifying a proof does not necessarily mean it correctly solves the original user prompt. There were cases where a proof was syntactically correct but logically incorrect relative to the problem statement. To resolve this, we added an extra verification step where, after Agda successfully compiles a proof, ChatGPT-4o is asked to confirm whether the proof correctly matches the original mathematical problem. When we sent this additional prompt to the API, we made sure to set the temperature setting to the lowest possible, ensuring an accurate response. This additional check significantly improves the reliability of verified proofs and ensures that they are not just formally correct but also aligned with the intended logic.

4.5 Key Decisions

Throughout the development of our system, several pivotal decisions were made to address unforeseen challenges and improve the reliability of proof verification. One of the most critical choices was switching from ChatGPT-4o Mini to the full ChatGPT-4o model after discovering that the Mini version consistently failed to generate Agda-compliant proofs due to its limited abilities compared to the full model. The increased computational cost was justified by the significantly improved proof accuracy and reduced failure rate.

To enhance verification accuracy, we also introduced an additional validation step after Agda successfully compiles a proof. Initially, our system considered an Agda-validated proof as correct, but we realized that syntactic correctness does not necessarily imply that the proof logically matches the original user prompt. To mitigate this, we implemented a step where ChatGPT-4o is asked to check whether the compiled proof correctly corresponds to the original mathematical statement. This additional verification ensures that the proof is not only valid but also relevant.

Another crucial decision was implementing a mechanism to detect redundant proof attempts and prevent the system from looping when ChatGPT-4o repeatedly generated the same incorrect proof structure. We introduced a check to compare failed proofs with previous attempts and provided the API with feedback to adjust its reasoning direction. While this does not entirely eliminate the issue, it significantly reduces the occurrence of infinite refinement loops, making the iterative proof verification process more efficient.

4.6 Tools Used

Throughout the development of our proof verification system, we utilized a variety of tools and frameworks to ensure efficiency, accuracy, and seamless integration between AI-driven proof generation and formal verification. Python served as the primary programming language for API development, enabling robust communication between ChatGPT-4o and Agda. The ChatGPT-4o API played a crucial role in generating mathematical proofs, which were then validated using Agda, our chosen formal proof verification engine based on dependent type theory. To create an intuitive and user-friendly interface, we employed PyQt5, which allowed us to design and implement a graphical user interface (GUI) that facilitates easy interaction with the system. PyCharm was used extensively for developing and testing Agda proofs, providing an efficient coding environment with syntax support and debugging capabilities. For system architecture and workflow visualization, we relied on Draw.io to design comprehensive system diagrams that illustrate the proof verification pipeline and user interactions. Given the computational demands of iterative proof refinement, we implemented multi-threading techniques to optimize performance, enabling parallel processing and minimizing delays in proof verification. Together, these tools and frameworks provided a solid foundation for developing a highly efficient and reliable

proof verification system, ensuring smooth integration between AI-assisted proof generation and formal mathematical validation.

4.7 Product Diagrams and GUI

We designed a desktop application to provide an intuitive user interface for engaging with the proof verification system.

4.7.1 Use-Case and Activity Diagrams

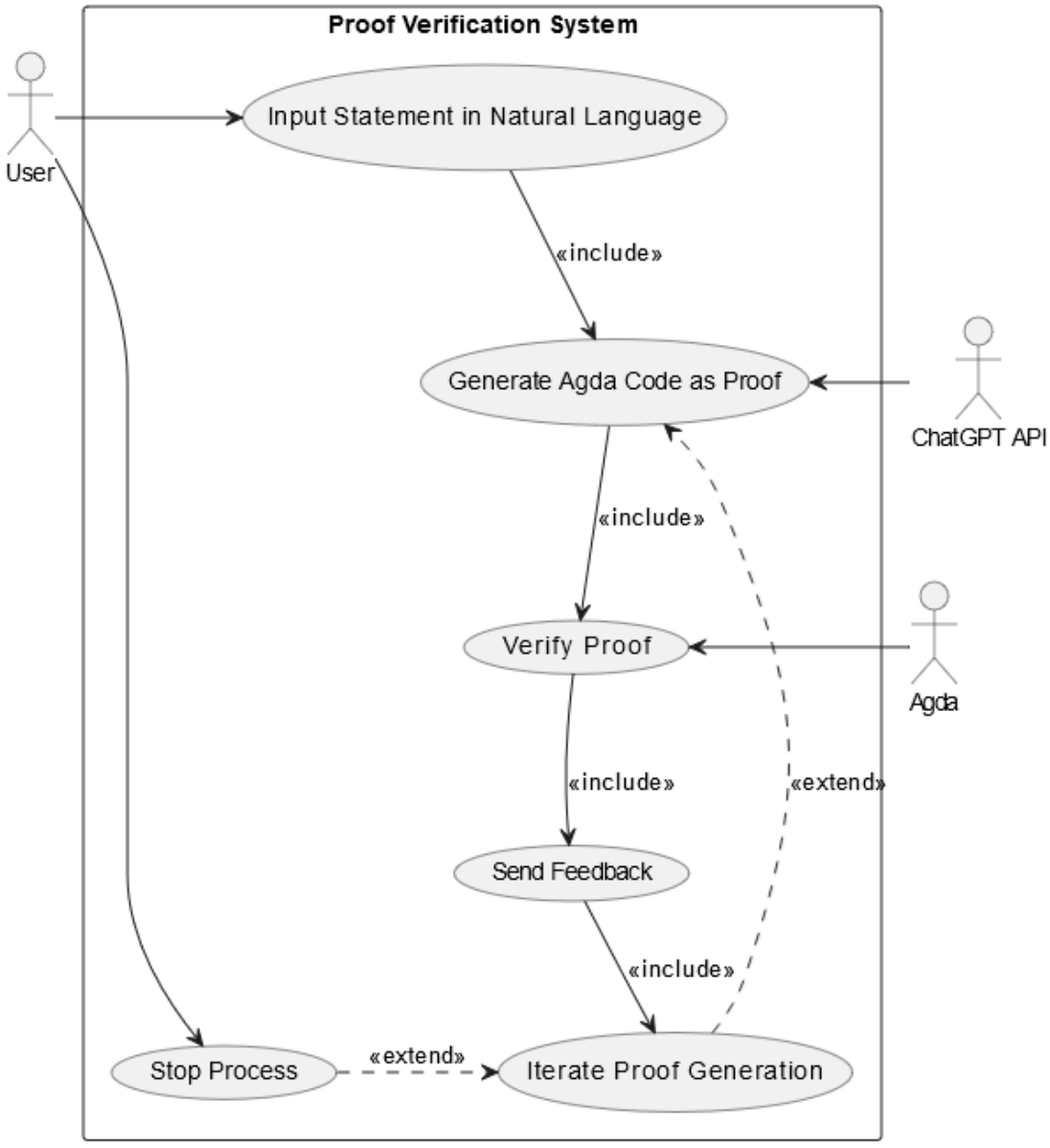

Figure 6. Use case diagram of the system. Created by http://www.draw.io

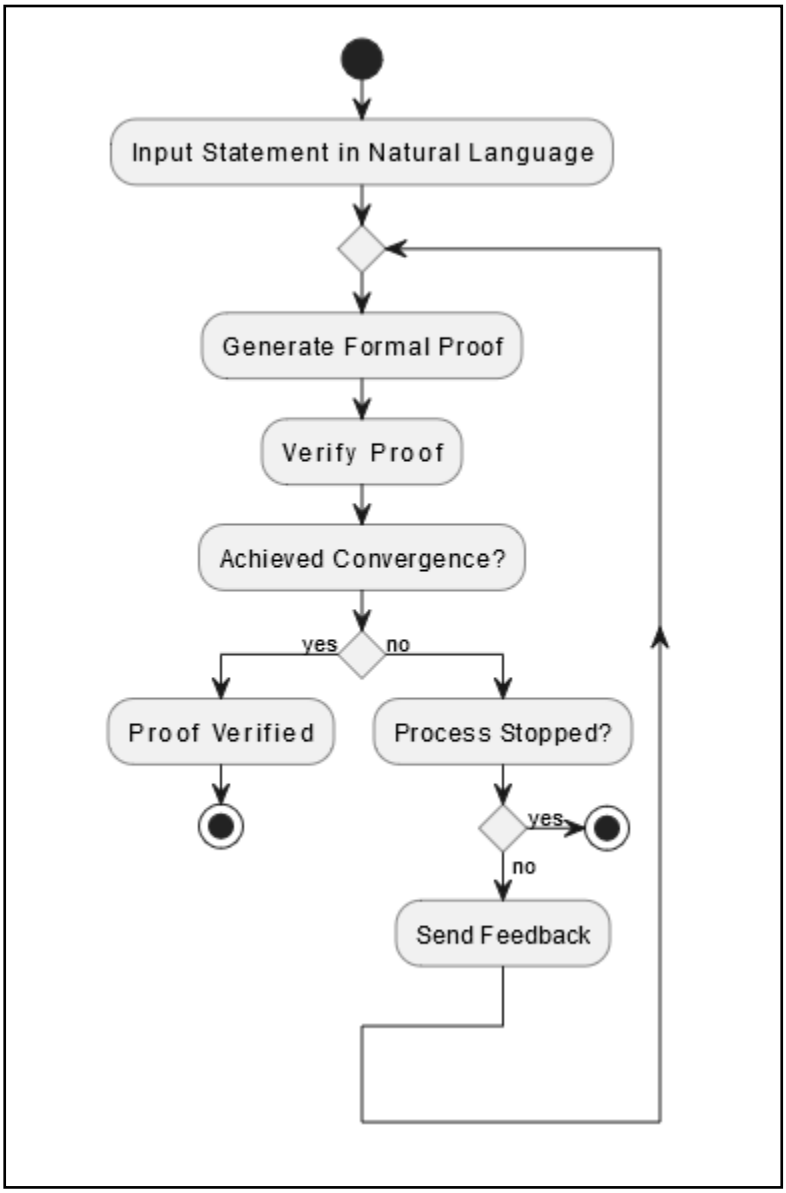

Figure 7. Activity diagram of the system. Created by http://www.draw.io

4.7.2 User Interaction with the UI

Our approach to designing the user interface (UI) was guided by the need for a seamless and intuitive experience that creates interaction with the proof verification system. Since our system is intended for researchers and mathematicians who may not be familiar with formal proof assistants, we prioritized clarity and accessibility. The UI, which was developed using PyQt5, provides a structured and user-friendly environment where users can submit mathematical problems in natural language, view ChatGPT-4o's generated proofs, track Agda verification results, and observe the iterative refinement process and its status in real-time. The UI of the system is demonstrated in Figure 8.

5 User and Maintenance Guides

5.1 User Guide

After successfully installing Agda, and ProofVerifer.exe, the application will be loaded in this state:

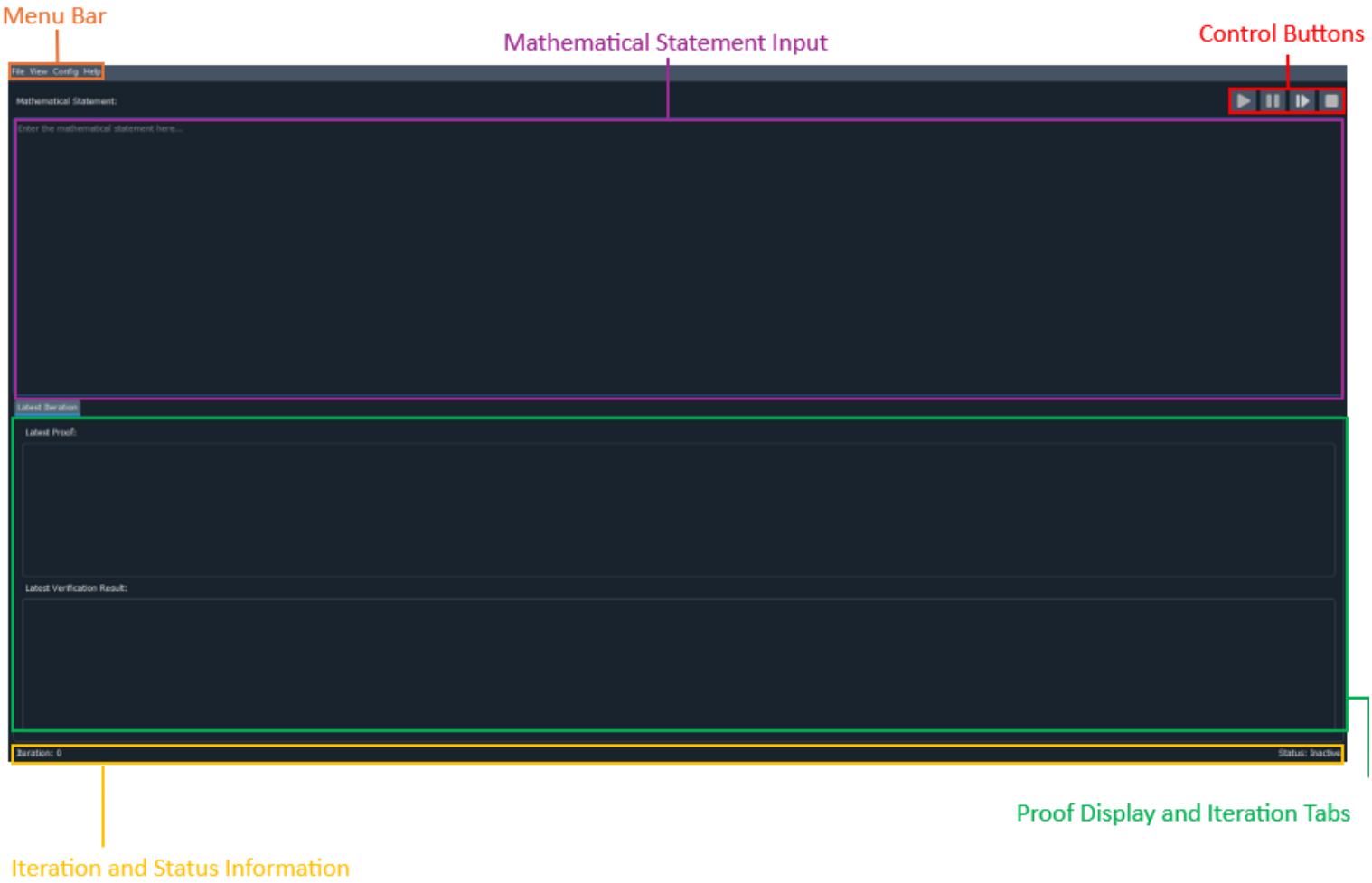

Figure 8. GUI of the system.

5.1.1 Menu Bar

1. File
    - Save to PDF

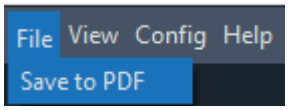

Figure 9. Save to PDF button

Downloads automatically to downloads folder a summary of the entire run, highlighting the number of iterations, the result, the original statement and each iteration separately.

2. View
    - Enable Dark Mode

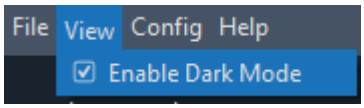

Figure 10. Enable dark mode button

Toggles the appearance, such as enabling or disabling dark mode (defaulted to dark mode, chosen mode will be saved in configuration for future uses).

3. Config
    - Set Iteration Limit

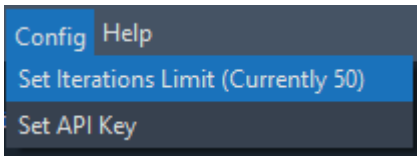

Figure 11. Set iteration limit button

Sets iteration limit for proof verification (Will be saved under <USER>\ProofVerifier for any operation system).

- Set API Key

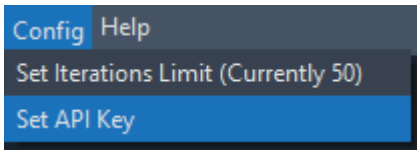

Figure 12. Set API key button

Mandatory step for running the verifier

validates OpenAI API Key (Will be saved under <USER>\ProofVerifier for any operation system).

4. Help
   - User Help

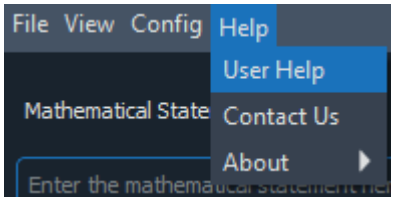

Figure 13. User help button

Shows the current PDF document, offers guidance on using the application.

   - Contact Us

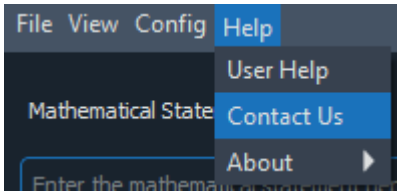

Figure 14. Contact us button

Shows the current PDF document, offers guidance on using the application.

- About

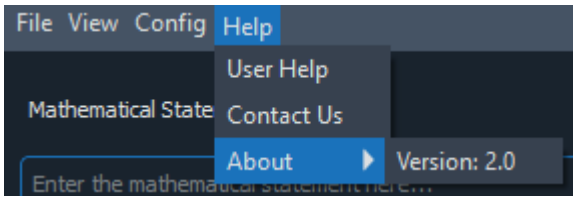

Figure 15. About button

Displays current system version and credits.

5.1.2 Mathematical Statement Input

In this section, the user can input a mathematical proof, and our system will try to validate its correctness by iteratively refining the proof using ChatGPT's API.

Figure 16. Mathematical statement input

Once the user will input a statement for verification, the "Verify" button will be enabled.

After hitting verify, the iterative proof verification process will begin.

5.1.3 Control Buttons

Proof Verifier has 4 control buttons for best user experience.

1. Verify Button – Initiate the Verification (enabled when verification process is inactive and when the there's a statement to verify).

Figure 17. Verify button

2. Pause Button – Pause the Verification (Will be paused at the end of the current iteration, enabled when the verification process is active).

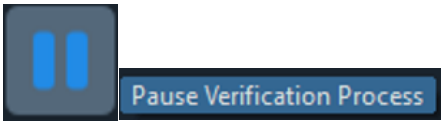

Figure 18. Pause button

3. Resume Button – Resume the Verification (enabled only when the verification process is paused).

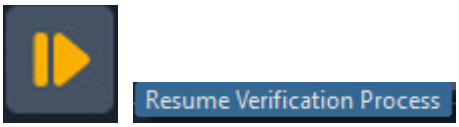

Figure 19. Resume button

4. Stop Button – Stop the iteration (Will be stopped at the end of the current iteration, enabled when the verification is paused or active).

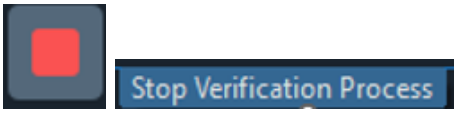

Figure 20. Stop button

5.1.4 Proof Display and Iteration Tabs

To view the progress of the verification process, proof verification provides a Proof Section (Agda code), Verification Result (feedback from Agda compiler), tabs for each iteration.

1. Proof Section – In this section, the user can see the proof generated by ChatGPT API in the Latest iteration.

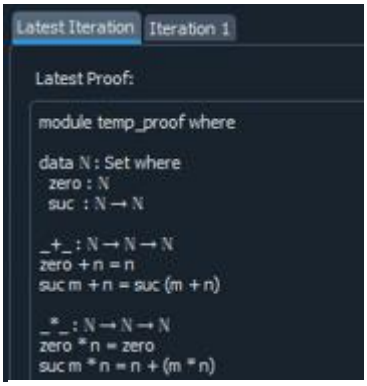

Figure 21. Proof section

2. Verification Result - In this section, the user can see the feedback from Agda compiler in the Latest iteration.

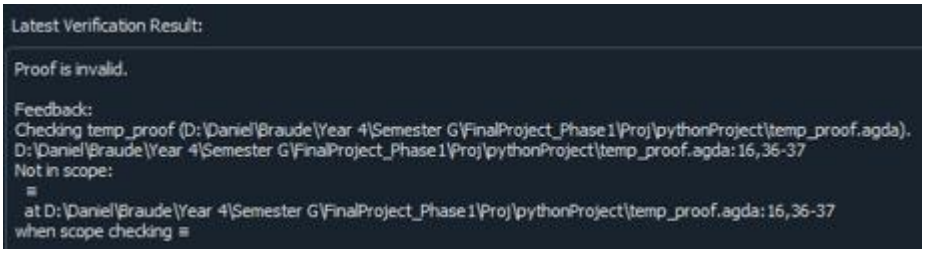

Figure 22. Verification result

3. Tabs – Each refinement will get its own iteration tab with the addition of a latest tab which will always contain the latest refinement.

Figure 23. Tabs

5.1.5 Iterations and Status Information

At the bottom of the screen, there is information about the current iteration and the status of the tool.

1. Iteration Info – displayed at the bottom left side of the screen.

Figure 24. Iteration info

2. Tool Status – displayed at the bottom right side of the screen, the following statuses exist:
    a. "Inactive" – When the verification process is finished or hasn't started.
    b. "Active" – When the verification process is active.
    c. "Pausing..." – When the verification process is being paused.
    d. "Paused" – When the verification process has been paused.
    e. "Stopping..." – When the verification process is being stopped.
    f. "Stopped" – When the verification process has been stopped.

5.1.6 Iterations and Status Information

1. Enter a mathematical statement in the input box.

2. Click the Verify button to initiate proof verification.

3. Monitor the iterative process through the tabs and status label.

4. Pause, resume, or stop the process as needed.

5. View results in the Latest Iteration tab or review detailed feedback in previous iteration tabs or alternatively download a pdf file with a detailed summary of the run.

* A valid proof has been found – A success message will be displayed.

* Iterations limit has been reached – A failure message will be displayed.

5.2 Maintenance Guide

To use ProofVerifier, installing Agda and adding it to system PATH is mandatory. To install Agda, follow these steps:

5.2.1 Installing Agda

Agda is a dependently typed functional programming language and proof assistant. Follow these instructions to install it:

1. Visit the official Agda installation guide: Agda Installation Instructions.

    - We recommend following **Option 1: Install Agda as a Haskell Package**.
    - Non-windows users need to make sure to install zlib and ncurses dependencies. For such cases, run this command in your terminal: `apt-get install zlib1g-dev libncurses5-dev`
    - Windows users can skip to **Install GHC and Cabal through GHCup**.

2. Follow the steps specific to your system:

    - Install GHC and Cabal (Cabal executable can usually be found in C:\ghcup\bin\cabal.exe).

- Use cabal install Agda to install Agda (Agda executable can usually be found in C:\cabal\bin\agda.exe).

3. Verify the installation by running the following command in your terminal:
   ```
   cd <AGDA EXECUTABLE FOLDER HERE>
   agda --version
   ```
   You should see the version number if Agda is installed correctly.

5.2.2 Adding Agda to System Path

To ensure that Agda is accessible from anywhere on your system:

1. Open the **Start Menu** and search for Settings
2. Select **System**
3. In the **System** window, select **About**.
4. Under **About**, find and select Advanced system settings.
5. In the **System Properties** window, click the Environment Variables button.
6. Under **System Variables**, find and select the Path variable, then click Edit.
7. Click **New** and add the path to Agda's executable (e.g., C:\cabal\bin\ - location of agda.exe file).
8. Click **OK** to save the changes.
9. Restart your terminal and verify Agda is in the path by typing: `agda --version`

5.2.3 Setting Up the Development Environment

- Set up the Python Integrated Development Environment (IDE) on your workstation. PyCharm, which offers excellent support for Python projects, including PyQt5 apps, is the program we suggested
- Clone the project repository from the terminal, using the repository URL
- Install required libraries from the terminal, by running the following command to install all the required libraries: "pip install -r requirements.txt"

5.2.4 Running the Application

After the previous steps have been successfully completed, the IDE is ready for work and the application is ready for use. To start the iterative process, an API Key must be provided to the system, through **Config – Set API Key**.

5.3 Example Proof

In this example, we will see a successful proof for the statement “a prime number is a natural number greater than 1 that is not a product of two smaller natural numbers “

Figure 25. Example refinement process

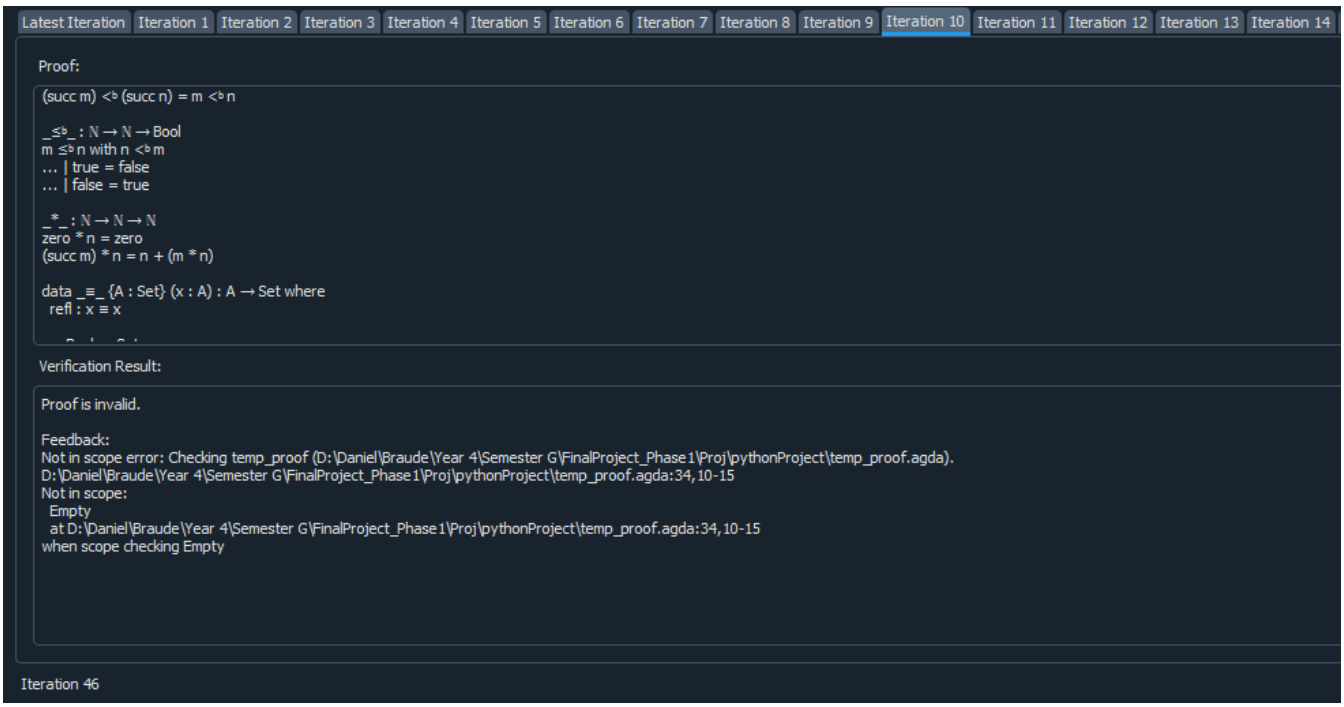

Figure 26. Example refinement process

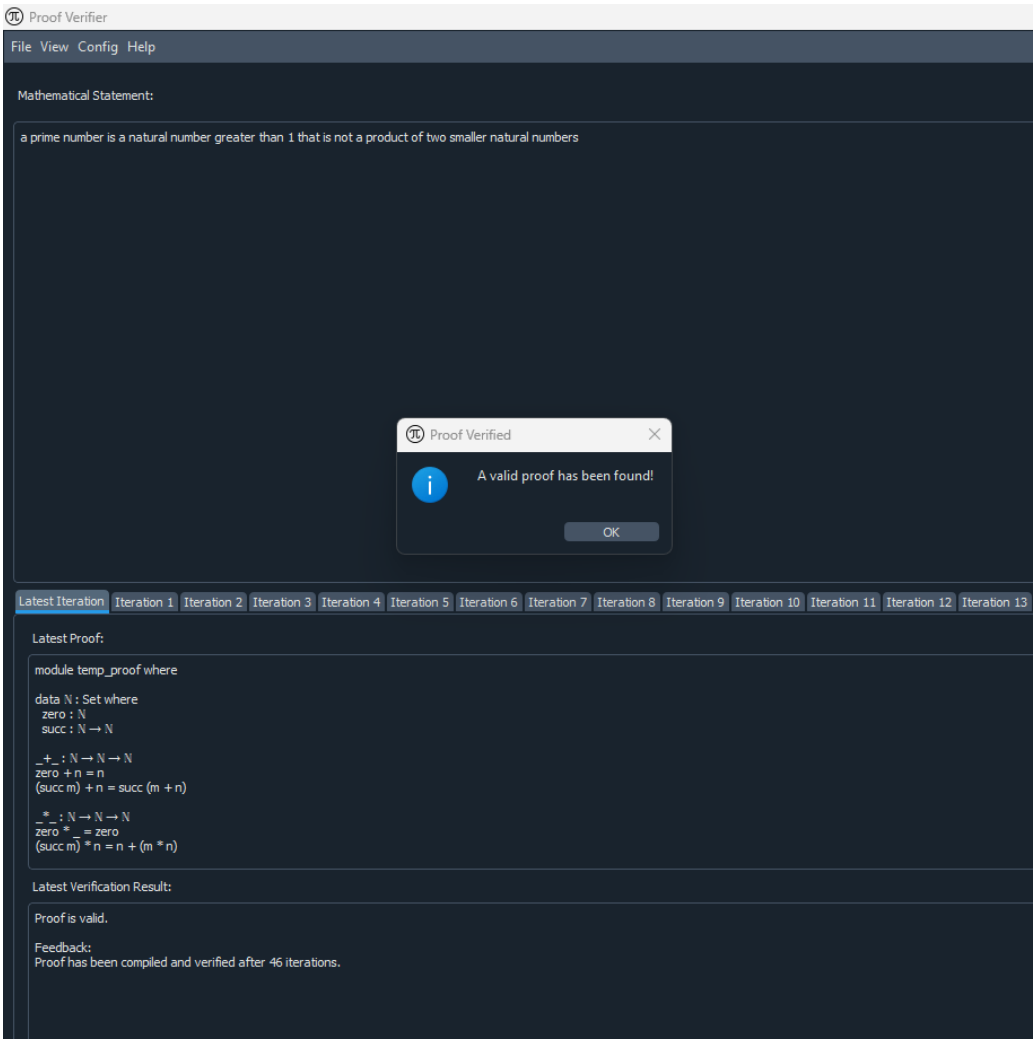

Figure 27. End of refinement process

In Figure 28, we can see ChatGPT – 4o's solution and proof of the inserted statement.

```
module temp_proof where

data ℕ : Set where
  zero : ℕ
  succ : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + n = n
(succ m) + n = succ (m + n)

_*_ : ℕ → ℕ → ℕ
zero * _ = zero
(succ m) * n = n + (m * n)

data Bool : Set where
  true  : Bool
  false : Bool

_<ᵇ_ : ℕ → ℕ → Bool
zero <ᵇ zero = false
zero <ᵇ (succ n) = true
(succ m) <ᵇ zero = false
(succ m) <ᵇ (succ n) = m <ᵇ n

_≤ᵇ_ : ℕ → ℕ → Bool
m ≤ᵇ n with n <ᵇ m
... | true = false
... | false = true

data _≡_ {A : Set} (x : A) : A → Set where
  refl : x ≡ x

_∨_ : Bool → Bool → Bool
true  ∨ _     = true
_     ∨ true  = true
false ∨ false = false

¬ : Bool → Bool
¬ true = false
¬ false = true

data Prime : ℕ → Set where
  prime : ∀ {n : ℕ} →
          (succ zero <ᵇ n) ≡ true →
          (∀ {d k : ℕ} →
            (succ d * succ k) ≡ n →
            ¬ ((succ d <ᵇ n) ∨ (succ k <ᵇ n)) ≡ true) →
          Prime n
```

Figure 28. API proof that was verified by Agda

6 Testing and Evaluation

To ensure the reliability, accuracy, and efficiency of our proof verification system, we conducted a comprehensive testing process. The system was evaluated based on its ability to correctly process natural language inputs, generate valid Agda proofs, handle edge cases, and optimize performance. Testing was divided into three key categories: functional testing, automated testing, and user acceptance testing (UAT).

6.1 Test Scenarios

The test cases outlined in Table 1 are designed to evaluate the core functionalities of the system, ensuring that it accurately processes natural language inputs, generates correct proofs, and handles various edge cases. Each test case simulates real-world usage scenarios to verify the reliability of the system.

Table 1: Suggested test cases for our program validation.

Test Case	Test Case Description	Expected Outcome
1	Input a valid natural language mathematical statement into ChatGPT-4o	ChatGPT generates Agda code as proof for verification.
2	Proof verification fails due to incorrect input	The system provides feedback, and ChatGPT refines the proof
3	Input a large and complex proof statement	The system efficiently handles the input and completes verification within an acceptable timeframe
4	Attempt to verify an invalid or incomplete proof	The system detects the errors, provides feedback, and does not falsely confirm validity
5	User interrupts the iterative refinement process	The system safely halts the refinement process, saving the current state of the proof
6	The maximum iteration limit is reached	The system safely halts the refinement process, saving the current state of the proof

6.2 Automated Testing

To verify individual system components, we implemented automated unit and integration testing using predefined mathematical statements and proofs. This allowed

us to validate the interactions between ChatGPT-4o, Agda, and the API, ensuring seamless communication and reliable performance.

6.2.1 Unit Testing

We conducted unit tests to verify the correctness of individual parts of our system, including API functionality, ensuring accurate translation of proofs between ChatGPT-4o and Agda, ChatGPT-4o output consistency, verifying that ChatGPT-4o generates proofs in an Agda-compatible format, and error-handling mechanisms to test the system's ability to detect invalid proofs and provide meaningful feedback.

6.2.2 Integration Testing

We performed integration tests to ensure smooth interactions between proof generation, proof verification, and iterative refinement. This involved testing the flow from ChatGPT-4o generating an initial proof to Agda verifying or rejecting it, followed by refinement through the API. The goal was to confirm that the system correctly processed errors and refinement suggestions while maintaining efficient communication between components. The iterative process was validated to ensure that it refined proofs until convergence, iterations limit reached or stopped by user.

6.2.3 Performance Testing

To assess efficiency and the performance of the system, we measured response time for different proof sizes and complexities, monitored memory and CPU usage during proof refinement. Given the iterative nature of proof verification, we implemented multithreading techniques to reduce response times. These optimizations significantly improved the system's ability to handle complex proofs while maintaining stability and efficiency.

6.3 User Acceptance Testing (UAT)

To validate the system's usability and accuracy in real-world applications, we conducted User Acceptance Testing (UAT) to ensure the system meets user expectations. The feedback focused on:

1. Accuracy of Proof Verification: Assessing the correctness of the verified proofs.
2. Ease of Use: Evaluating the intuitiveness of the user interface and refinement process.
3. Error Handling: Ensuring that the system provides clear and actionable feedback for wrong proofs.

6.4 Overall System Performance Evaluation

The success of our system is evaluated based on accuracy, reliability, efficiency, scalability, and user experience. Accuracy is determined by the system's ability to generate Agda-compliant proofs and iteratively refine incorrect ones, though some proofs may fail to converge. Reliability is assessed through structured refinement, ensuring that verification aligns with formal logic. Scalability is tested by handling increasingly complex proofs while maintaining verification consistency, and efficiency is measured by achieving verification within a reasonable number of iterations.

User Acceptance Testing validates the system's usability, focusing on intuitiveness, proof refinement clarity, and structured error feedback. The extra validation step ensures Agda-verified proofs logically match the original problem, preventing syntactically valid but incorrect solutions. The redundant proof detection mechanism improves efficiency by reducing repetitive refinement cycles, though it does not always eliminate non-convergence.

7 Results

Our proof verification system successfully integrates ChatGPT-4o and Agda, demonstrating that AI-assisted formal proof verification is feasible. Testing confirmed that the system can generate Agda-compliant proofs and refine incorrect ones iteratively. The extra validation step ensured that Agda-verified proofs matched the original problem, reducing incorrect confirmations. The redundant proof detection mechanism helped prevent repeated refinement failures, improving efficiency.

However, not all proofs converged within the iteration limit. Highly abstract mathematical statements and ambiguous feedback from Agda sometimes led to lack of progress in refinement. While structured refinement improved success rates, some cases remained unresolved, indicating a need for better adaptation in AI-driven proof reasoning. Despite these challenges, the system met its core objectives: enabling automated proof generation, integrating formal verification, refining AI-generated proofs through iterative feedback and increasing reliability in AI generated proofs.

8 Insights

Several key insights emerged from our research. First, while ChatGPT-4o can generate formal proofs, it often struggles with nuanced logical refinements, leading to non-converging verification loops. This highlights the limitations of LLMs in structured mathematical reasoning. Second, Agda's strict syntax ensures formal correctness, but logical correctness still requires additional validation, reinforcing the need for post-verification checks beyond Agda’s approval.

Another insight is that proof refinement efficiency depends heavily on prompt engineering. Minor modifications to feedback structure significantly impacted refinement quality. Additionally, our redundant proof detection mechanism reduced unnecessary iterations, but it did not completely resolve repetitive failure patterns, indicating that AI reasoning requires deeper contextual awareness for iterative corrections.

Lastly, while our system bridges AI-driven proof generation with formal verification, it is clear that no single proof assistant is universally optimal. Exploring alternative verification frameworks, newer versions of Agda compiler, stronger LLM models could provide broader adaptability in future research.

9 Conclusion

This research demonstrates that integrating ChatGPT-4o with Agda significantly enhances AI-assisted proof verification, bridging the gap between large language models and formal mathematical reasoning. Our system successfully automates proof generation, refines incorrect proofs through iterative feedback, and ensures formal correctness using Agda. The addition of a post-verification check further strengthens reliability by ensuring that syntactically valid proofs also align with the original problem statement.

However, challenges remain. Proof convergence is not guaranteed, as some complex mathematical statements fail to refine within the iteration limit, particularly when Agda's feedback lacks sufficient clarity. While our redundant proof detection mechanism improves refinement efficiency, it does not fully prevent repetitive failures, highlighting the limitations of LLMs in deep logical adjustments. Despite these constraints, our approach successfully increases the trustworthiness of AI-generated proofs, making AI-driven formal verification a more practical and scalable tool for mathematical research.

Our findings confirm that AI can play a meaningful role in theorem proving, but further refinements in proof refinement strategies and verification frameworks are necessary to maximize effectiveness. The work presented here lays a strong foundation for future research in LLM-driven proof validation, reinforcing the potential of AI in formal mathematics.

10 Bibliography

[1] Azaria, A., Azoulay, R., & Reches, S. (2024). ChatGPT is a remarkable tool—for experts. *Data Intelligence*, *6*(1), 240-296. https://doi.org/10.1162/dint_a_00235

[2] Janßen, C., Richter, C., & Wehrheim, H. (2024, April). Can ChatGPT support software verification? In *International Conference on Fundamental Approaches to Software Engineering* (pp. 266-279). Cham: Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-57259-3_13

[3] Martin-Löf, P., & Sambin, G. (1984). *Intuitionistic type theory* (Vol. 9, p. 136). Naples: Bibliopolis. https://archive-pml.github.io/martin-lof/pdfs/Bibliopolis-Book-retypeset-1984.pdf

[4] Pelayo, Á., & Warren, M. (2014). Homotopy type theory and Voevodsky's univalent foundations. *Bulletin of the American Mathematical Society, 51*(4), 597-648. https://doi.org/10.1090/S0273-0979-2014-01456-9

[5] Plevris, V., Papazafeiropoulos, G., & Rios, A. J. (2023). Chatbots put to the test in math and logic problems: A preliminary comparison and assessment of ChatGPT-3.5, ChatGPT-4, and Google Bard. *arXiv preprint arXiv:2305.18618*. https://doi.org/10.48550/arXiv.2305.18618

[6] Program, T. U. F. (2013). Homotopy type theory: Univalent foundations of mathematics. arXiv preprint arXiv:1308.0729. https://doi.org/10.48550/arXiv.1308.0729

[7] Shakarian, P., Koyyalamudi, A., Ngu, N., & Mareedu, L. (2023). An independent evaluation of ChatGPT on mathematical word problems (MWP). https://doi.org/10.48550/arXiv.2302.13814

[8] Sitnikovski, B. (2023). *Introduction to dependent types with Idris: Encoding program proofs in types*. Apress. https://doi.org/10.1007/978-1-4842-9259-4

[9] Stump, A. (2016). *Verified functional programming in Agda*. Morgan & Claypool. https://doi.org/10.1145/2841316

11 GitHub Repository

https://github.com/daniarmag/ProofVerifier